CS 185/285 Assignment 2: Policy Gradients — Report

All experiments ran on CPU with wandb in offline mode. Both are set through environment variables only, so the training commands themselves match the assignment handout:

```
export CUDA_VISIBLE_DEVICES="" WANDB_MODE=offline OMP_NUM_THREADS=2
```

Each configuration was run once (seed 1, and the environment itself is not seeded), so the comparisons below are single-run results and small gaps between configurations may just be noise. In every plot the x-axis is the number of environment steps (`Train_EnvstepsSoFar`) and the y-axis is `Eval_AverageReturn`. The figures are produced by `uv run src/scripts/plot.py`.

Experiment 1: CartPole

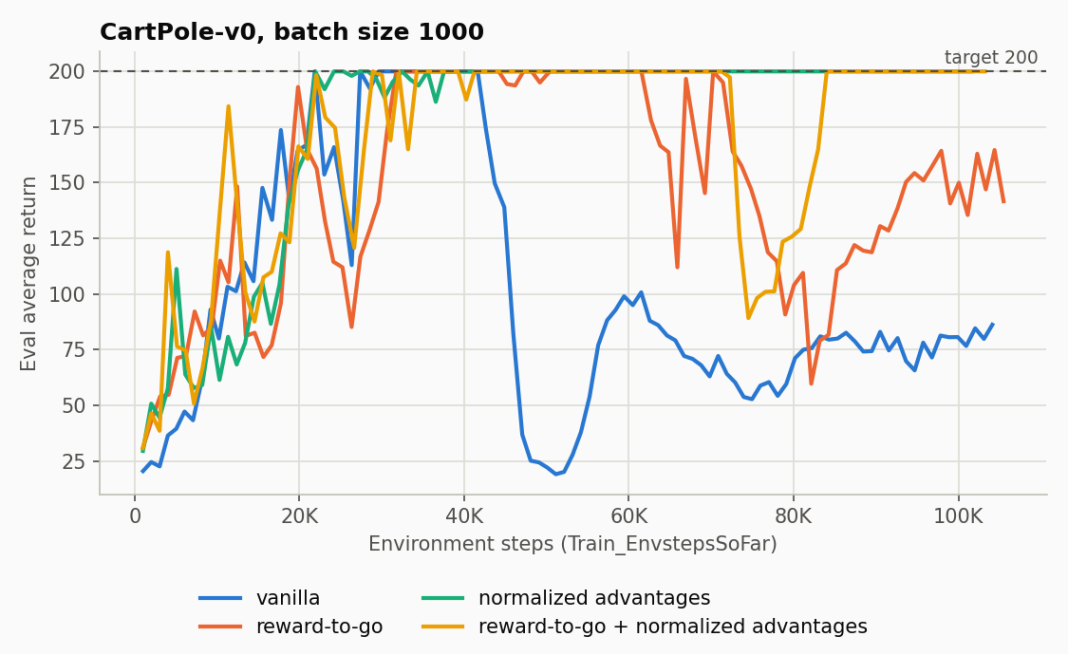


Figure 1: CartPole, small batch

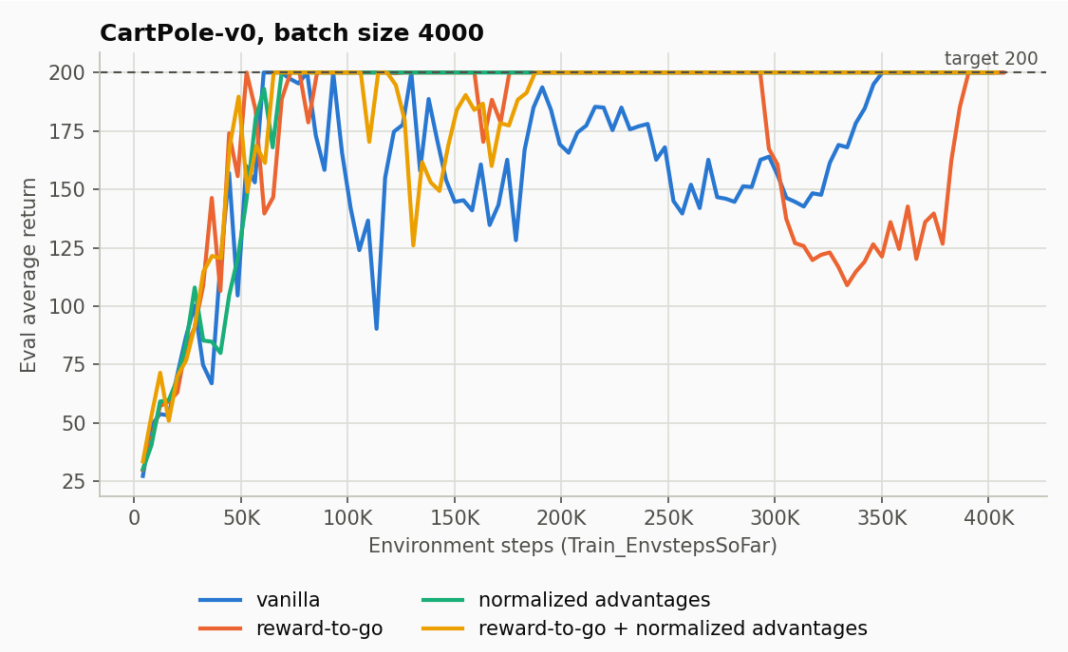


Figure 2: CartPole, large batch

Configuration	Mean return over all 100 evals	Mean over the last 50	Evals reaching 200
`cartpole`	99.6	73.6	15
`cartpole_rtg`	147.1	149.8	28
`cartpole_na`	176.5	200.0	72
`cartpole_rtg_na`	169.4	184.1	59
`cartpole_lb`	159.6	173.9	21
`cartpole_lb_rtg`	169.6	169.3	56
`cartpole_lb_na`	184.1	200.0	84
`cartpole_lb_rtg_na`	181.8	200.0	68

All eight configurations reached the maximum return of 200 at least once.

Which value estimator performs better without advantage normalization? Reward-to-go. With the small batch, the mean return over training is 147.1 versus 99.6. With the large batch it is 169.6 versus 159.6, and reward-to-go reaches 200 far more often (56 versus 21 evals). The small-batch trajectory-centric (vanilla) run collapses after about 40K steps and stays around 75 afterwards.

Why is one generally preferred? Reward-to-go exploits causality: the action at time t cannot affect rewards before t, so those terms have zero expected gradient and only add variance. Dropping them keeps the estimator unbiased while lowering its variance. In addition, the discount is applied relative to the current time step, so states late in the trajectory are not disproportionately ignored.

Did advantage normalization help? Yes, and it was the most influential factor in this experiment. All four `-na` runs average at least 184 over the second half of training, and three of them score 200 on every one of the last 50 evals; all four runs without `-na` show clear drops in the second half. CartPole rewards are all positive, so without normalization every action's probability is pushed up, just by different amounts. After normalization, below-average actions are pushed down, and the gradient magnitude no longer depends on the reward scale. In these single runs, `-na` alone did slightly better than `-rtg -na`; I consider that gap to be within noise.

Did the batch size make an impact? Yes. The large-batch (4000) curves are smoother, and the two unnormalized configurations do much better (vanilla second-half mean 173.9 versus 73.6). The cost is sample efficiency: the small batch first reaches 200 after about 22K steps at best, while the large batch needs about 50–70K steps, because each gradient update consumes four times as much data.

Commands:

```
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 --exp_name cartpole
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -rtg --exp_name
cartpole_rtg
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -na --exp_name
cartpole_na
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 1000 -rtg -na --exp_name
cartpole_rtg_na
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 --exp_name cartpole_lb
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -rtg --exp_name
cartpole_lb_rtg
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -na --exp_name
cartpole_lb_na
uv run src/scripts/run.py --env_name CartPole-v0 -n 100 -b 4000 -rtg -na --exp_name
cartpole_lb_rtg_na
```

Experiment 2: HalfCheetah with a neural network baseline

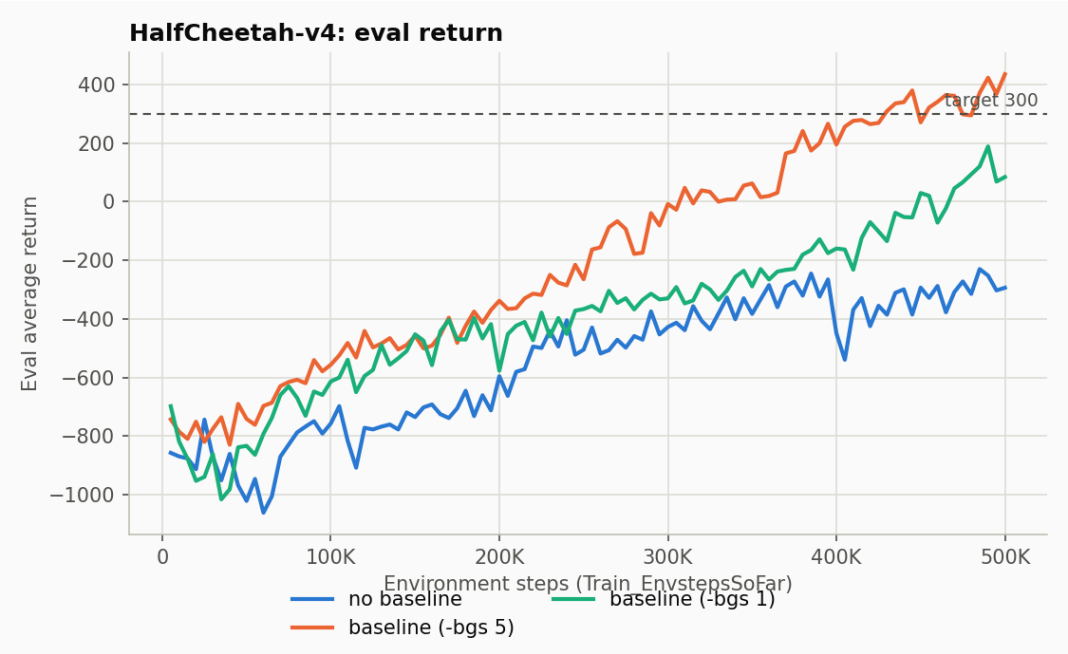


Figure 3: HalfCheetah eval return

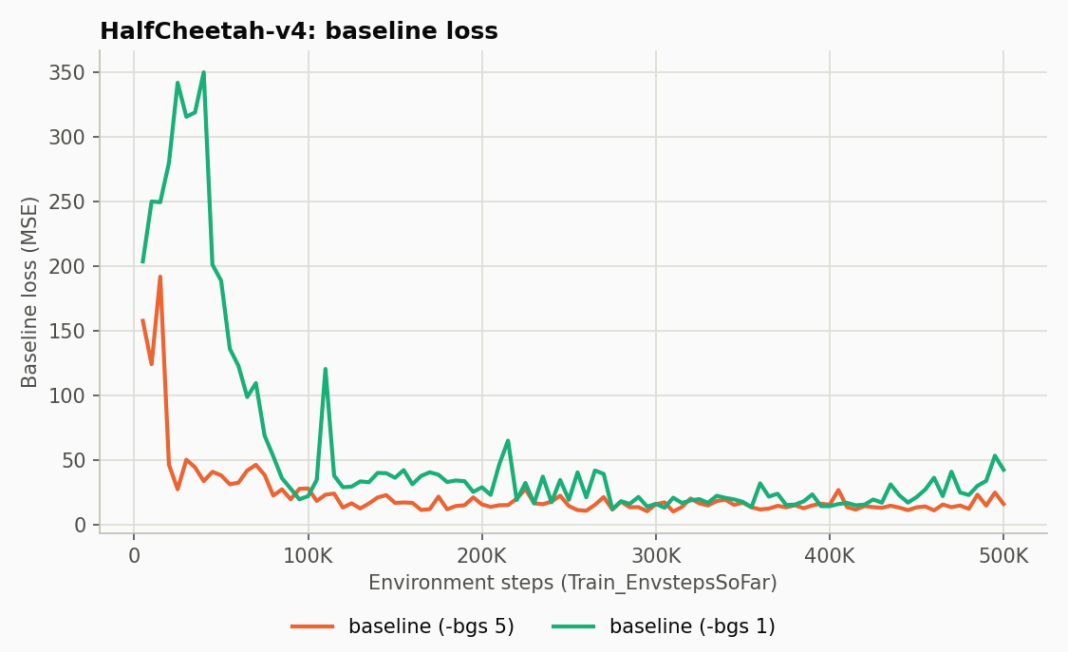


Figure 4: HalfCheetah baseline loss

Configuration	Final return	Mean of last 10	Max
`cheetah` (no baseline)	−293.6	−296.6	−229.7
`cheetah_baseline` (`-bgs 5`, submitted run)	435.5	358.1	435.5
`cheetah_baseline_bgs1` (`-bgs 1`)	84.5	59.6	188.9

The submitted baseline run ends at 435.5, above the threshold of 300.

About reruns: `cheetah_baseline` was run four times in total. The first run ended at only −59.0 (max 112.1) and missed the threshold. Three reruns then ended at 455.7, 435.5 and 449.7, all above it. The submitted run is the one of these with the highest mean over its last 10 evals. Logs of all four runs are kept (the submitted one in `exp/`, the others in `exp_extra/`). This also shows how high the variance of this setting is.

Effect of fewer baseline gradient steps (`-bgs 5` → `-bgs 1`):

- (a) The baseline loss falls much more slowly. With `-bgs 5` it drops below 50 by about 20K steps. With `-bgs 1` it first rises to about 350 and only reaches the same level at about 85K steps, and it stays somewhat higher and noisier afterwards (about 43 versus 16 at the end).
- (b) The policy performs worse: it ends at 84.5, still better than −293.6 without a baseline, but below the three `-bgs 5` runs that passed (435–456). The worse the baseline fits, the less variance it removes. However, `-bgs 1` was run only once and one `-bgs 5` run also reached only −59, so this conclusion is tentative.

Commands:

```
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg \
  --discount 0.95 -lr 0.01 --exp_name cheetah
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg \
  --discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 5 --exp_name cheetah_baseline
uv run src/scripts/run.py --env_name HalfCheetah-v4 -n 100 -b 5000 -eb 3000 -rtg \
  --discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 1 --exp_name
cheetah_baseline_bgs1
```

Experiment 3: LunarLander with GAE

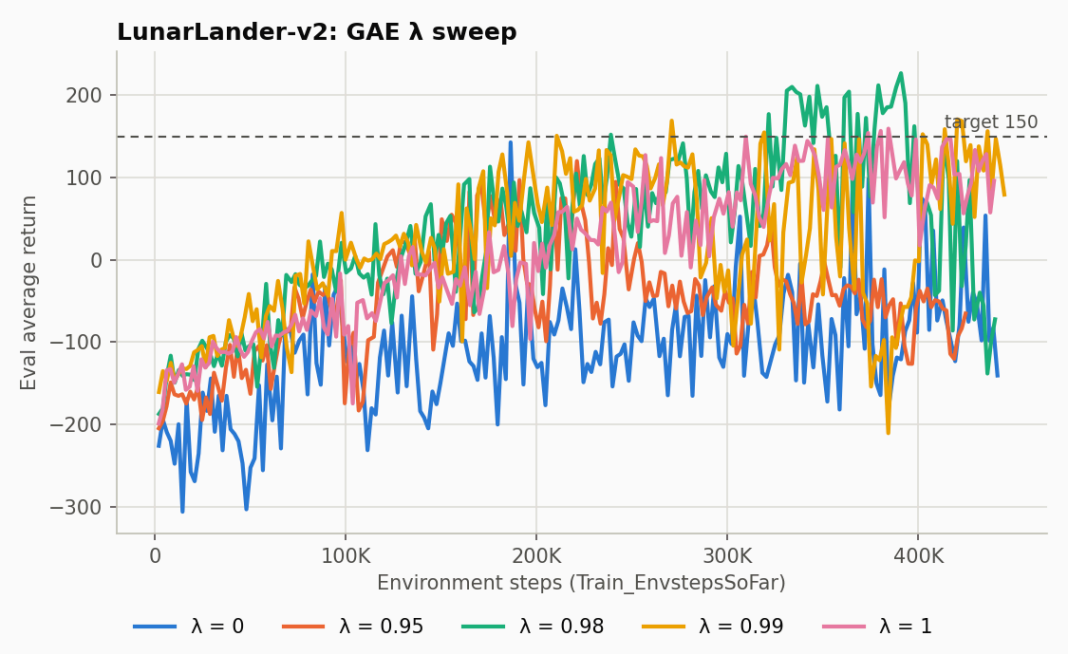


Figure 5: LunarLander λ sweep

λ	Max return	Mean over the last 100 evals	Evals above 150
0	142.6	−80.8	0
0.95	119.9	−41.5	0
0.98	**226.5**	**94.1**	24
0.99	170.6	51.4	8
1	159.2	83.9	3

The best run (λ = 0.98) peaks at 226.5, above the threshold of 150; λ = 0.99 and λ = 1 also exceeded it.

How λ affected performance: The three runs with larger λ (0.98, 0.99, 1) are clearly better, with 0.98 the best. λ = 0 is the worst: it learns the slowest, oscillates the most, and spends most of training around −100. λ = 0.95 sits in between and never exceeds 150. λ = 1 peaks lower than 0.98 but is the most stable late in training (its second-half minimum is −15, while every other run dropped below −125). All runs still fluctuate strongly at the end of training.

What $\lambda = 0$ and $\lambda = 1$ correspond to: With $\lambda = 0$ the advantage is the one-step TD error $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$, which has the lowest variance but relies entirely on the critic and is heavily biased when the critic is inaccurate. With $\lambda = 1$ it equals the Monte Carlo reward-to-go minus the baseline $V(s_t)$, which is unbiased but has the highest variance. On LunarLander the critic only gets 5 gradient steps per iteration and is not accurate, so the bias of $\lambda = 0$ makes it the worst; $\lambda = 1$ learns but is noisier; a λ slightly below 1 (0.98) gives the best trade-off.

Command (`<lambda>` is 0, 0.95, 0.98, 0.99 and 1):

```
uv run src/scripts/run.py --env_name LunarLander-v2 --ep_len 1000 --discount 0.99 \
  -n 200 -b 2000 -eb 2000 -l 3 -s 128 -lr 0.001 --use_reward_to_go --use_baseline \
  --gae_lambda <lambda> --exp_name lunar_lander_lambda<lambda>
```

Experiment 4: InvertedPendulum hyperparameter tuning

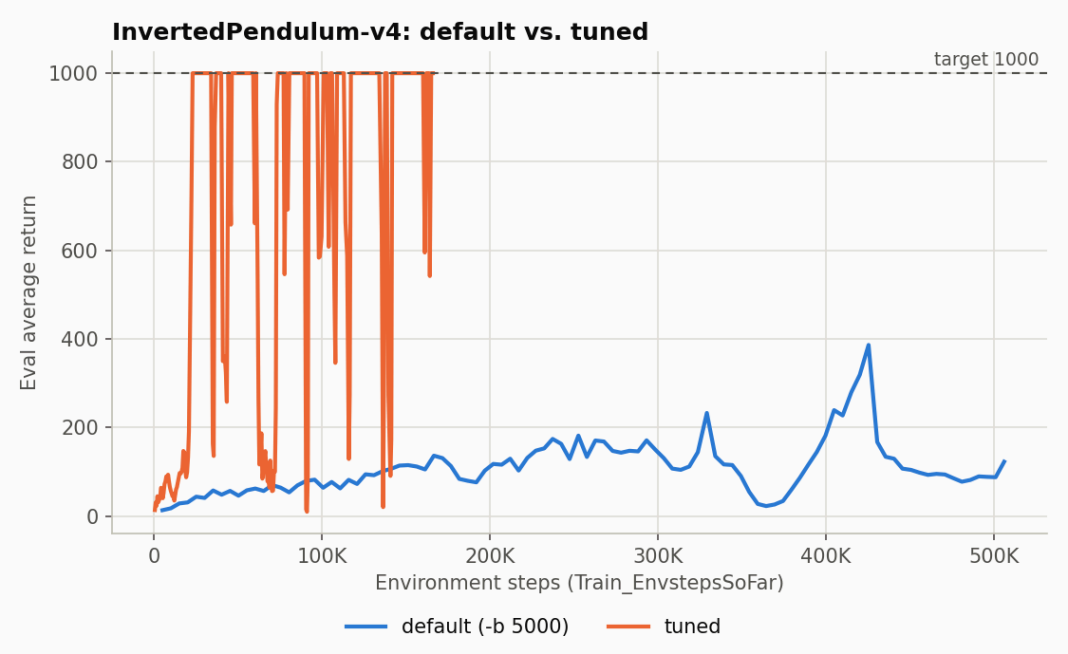


Figure 6: InvertedPendulum, default vs. tuned

Best configuration:

```
uv run src/scripts/run.py --env_name InvertedPendulum-v4 -n 200 -b 500 -eb 1000 \
  -lr 0.02 -rtg -na --discount 0.99 --use_baseline --gae_lambda 0.98 --exp_name
pendulum_tC
```

Changes from the default: batch size 5000 ⟶ 500, learning rate 0.005 ⟶ 0.02, reward-to-go on, advantage normalization on, baseline with GAE ($\lambda = 0.98$) on, discount 1.0 ⟶ 0.99, and iterations 100 ⟶ 200.

This run first reaches a return of 1000 at **22,974 steps**, far inside the 100K-step requirement. The default setting peaks at only 386.3 over its full 500K steps and never reaches 1000. Note that the tuned policy is not stable: after reaching 1000 it briefly collapses and recovers several times (see the plot), and its last 10 evals average 913.7. Also, since an episode can last 1000 steps while the batch is only 500, the 200 iterations actually used about 166K steps.

Default command, for comparison:

```
uv run src/scripts/run.py --env_name InvertedPendulum-v4 -n 100 -b 5000 -eb 1000 --
exp_name pendulum
```

The five configurations I tried (all sharing `-eb 1000 -rtg -na --discount 0.99`) all reached 1000 within 100K steps:

Name	Other settings	Steps to first reach 1000	Mean of last 10 evals
`pendulum_tC`	`-b 500 -lr 0.02`, baseline + GAE 0.98	22,974	913.7
`pendulum_tA`	`-b 1000 -lr 0.01`, baseline + GAE 0.98	37,071	526.9
`pendulum_tD`	`-b 1000 -lr 0.02`, baseline + GAE 0.98	47,694	228.7
`pendulum_tE`	`-b 1000 -lr 0.01`, no baseline	53,868	505.9
`pendulum_tB`	`-b 2000 -lr 0.01`, baseline + GAE 0.98	91,183	605.0

Which hyperparameters mattered:

- **Batch size mattered most.** With otherwise identical settings, reducing the batch from 2000 to 1000 cut the steps to first reach 1000 from about 91K to 37K; reducing it further to 500 (with a higher learning rate) gave 23K. The default of 5000 wastes many samples on every update.
- **Reward-to-go and advantage normalization** are the common basis of every tuned run; these two plus a smaller batch (`pendulum_tE`) are already enough to reach 1000 at 54K steps.
- **Baseline + GAE** helped: at batch 1000 and learning rate 0.01, adding them moved the first 1000 from 54K to 37K steps.
- **Learning rate** had a mixed effect: at batch 1000, raising it from 0.01 to 0.02 was slower and less stable.

Each configuration was run only once, so the smaller gaps in this ranking may not be reliable.